Friday Debugging Challenge: SOLID Principles

The Scenario

You have been asked to review an existing **Employee Management System**.

The code works, but the previous developer put almost everything into one class.

Your task is to review the code and identify the design problems.

Your challenge

Do not immediately rewrite the code.

First:

1. Read through the code carefully.
2. Identify anything that looks difficult to maintain.
3. Identify which **SOLID principle(s)** are being violated.
4. Explain **why** you think it is a violation.
5. Suggest how you would refactor it.
6. Only then start changing the code.

```
class Employee {
  constructor(name, email, role, salary) {
    this.name = name;
    this.email = email;
    this.role = role;
    this.salary = salary;
  }

  calculatePay() {
    if (this.role === "Developer") {
      return this.salary + 2000;
    }

    if (this.role === "Manager") {
      return this.salary + 5000;
    }

    if (this.role === "Intern") {
      return this.salary;
    }
```

```
    return this.salary;
  }

  generateReport() {
    return `
      Employee: ${this.name}
      Email: ${this.email}
      Role: ${this.role}
      Salary: ${this.salary}
    `;
  }

  saveToDatabase() {
    console.log(`Saving ${this.name} to the database...`);
  }

  sendEmail(message) {
    console.log(`Sending email to ${this.email}: ${message}`);
  }

  promote() {
    if (this.role === "Developer") {
      this.role = "Senior Developer";
      this.salary += 3000;
    } else if (this.role === "Intern") {
      this.role = "Developer";
      this.salary += 2000;
    }
  }

  work() {
    if (this.role === "Developer") {
      return "Writing code...";
    }

    if (this.role === "Manager") {
      return "Managing the development team...";
    }

    if (this.role === "Intern") {
      return "Learning and assisting the team...";
    }

    return "Working...";
  }
}


class Developer extends Employee {
  work() {
    return "Writing code...";
  }

  sendEmail(message) {
    throw new Error("Developers cannot send emails");
  }
```

```
}


class Manager extends Employee {
  work() {
    return "Managing the development team...";
  }
}


class Intern extends Employee {
  work() {
    return "Learning and assisting the team...";
  }

  calculatePay() {
    throw new Error("Interns do not receive a salary calculation");
  }
}


function processEmployee(employee) {
  console.log(employee.work());
  console.log(employee.calculatePay());

  if (employee instanceof Developer) {
    console.log("Developer-specific processing");
  }

  if (employee instanceof Manager) {
    console.log("Manager-specific processing");
  }

  if (employee instanceof Intern) {
    console.log("Intern-specific processing");
  }
}
```

Your Tasks

Task 1 — Find the SOLID violations

Try to identify **at least four** SOLID violations.

For each one, answer:

> Which principle is being violated?

and:

What makes you think it is being violated?

Task 2 — Look at the `Employee` class

How many different responsibilities does `Employee` currently have?

Think about:

- calculating pay
- generating reports
- saving to a database
- sending emails
- promotions
- deciding how employees work

Ask yourself:

> **Should one class be responsible for all of these things?**

Task 3 — Look at `calculatePay()`

What happens if the company introduces:

Designer
Tester
Product Manager
DevOps Engineer

Would you have to modify `calculatePay()`?

Ask yourself:

> **What happens when new employee types are added?**

Task 4 — Look at the subclasses

Look carefully at:

class Developer extends Employee

and:

class Intern extends Employee

Then look at:

sendEmail()

and:

calculatePay()

Ask yourself:

> **Can every subclass actually be used wherever an `Employee` is expected?**

Task 5 — Look at `processEmployee()`

Why does this function need to know whether an employee is:

Developer
Manager
Intern

What happens when another employee type is introduced?

Ask yourself:

> **Should this function need to keep checking the concrete type of every employee?**
